

Problema 1: Analice el siguiente programa:

```
void function (int n) {
    int i, j, k, counter = 0;
    for (i = n/2; i <= n; i++) { ← O(n)
        for (j = 1; j + n/2 <= n; j++) { ← O(n)
            for (k = 1; k <= n; k = k*2) { ← O(log(n))
                counter++;
            }
        }
    }
}
```

a) Encuentre la complejidad de tiempo en notación Big-Oh. Muestre todo su procedimiento.

$i = n/2$ para el primer for
 $j = n/2$ para el segundo for
 $k = 2^{p-1}$
 $\log_2 n = p-1$
 $\log_2 n + 1 = p$ para el tercer for

Juntándolos:

$O(n^2 \log n)$

Problema 2: Analice el siguiente programa:

```
void function (int n) {
    if (n <= 1) return; ← O(1)
    int i, j;
    for (i = 1; i <= n; i++) { ← O(n)
        for (j = 1; j <= n; j++) { ← O(1)
            printf("Sequence\n");
            break;
        }
    }
}
```

a) Encuentre la complejidad de tiempo en notación Big-Oh. Muestre todo su procedimiento.

it va revisando solo 1 vez, entonces es constante
 $i = n$
 $j = 1$ por el break

Juntándolos: $O(n)$

Problema 3: Analice el siguiente programa:

```
void function(int n) {  
    int i, j;  
    for (i = 1; i <= n/3; i++) {  $\leftarrow O(n)$   
        for (j = 1; j <= n; j += 4) {  $\leftarrow O(n)$   
            printf("Sequence\n");  
        }  
    }  
}
```

a) Encuentre la complejidad de tiempo en notación Big-Oh. Muestre todo su procedimiento.

$$i = n/3$$

$$j = n/4$$

Juntándolos:

$$O(n^2)$$

Problema 4: Encuentre el mejor caso, caso promedio y peor caso del algoritmo de Búsqueda Lineal (Linear Search, Binary Search y Quick sort). Muestre todo su procedimiento.

Linear Search: este algoritmo va uno por uno comparando

Mejor caso



cuando el elemento que se está buscando está en la primera posición.

por ejemplo busco 1 y en la lista está ordenada así: {1, 2, 3, 4}

al ser solo un movimiento, $n=1$

$$O(1)$$

Caso promedio



tomando el caso de una lista con n elementos, realiza n comparaciones

y se ve así: $(n+1)/2$ por lo que el big-Oh es:

$$O(n)$$

Peor caso



en este caso, el elemento está en la última posición o directamente no se encuentra por lo que $n=n$ al revisar n veces la lista.

$$O(n)$$

Binary Search: en una lista ordenada, este algoritmo, en cada paso, compara el valor buscado con el elemento central y descarta la mitad hasta encontrar el elemento.

Mejor caso:



Al igual que con el Linear Search, si el primer elemento es el valor buscado $k=1$ entonces

$$O(1)$$

Caso promedio

↓

Debemos suponer de que cada búsqueda tiene la misma probabilidad de ser buscado. Por lo que se forman niveles, $1, 2, 4, \dots = 2^0, 2^1, 2^2$ por lo que $K = 2^{P-1} - 1$, despejando queda $\log_2(K+1) - 1 = P$, entonces

$$O(\log n)$$

Peor caso

↓

en este caso, el elemento está en el último nivel de búsqueda o no se encuentra. Por lo que se realiza el procedimiento hasta que ya no se puede dividir.

y queda $n/2^k = 1$ donde $k = \log_2 n$

$$O(\log n)$$

Quick Sort: En este algoritmo se selecciona un pivote, distribuye los elementos mediante una partición y ordena recursivamente los partes

Mejor caso

↓

Se dividen los elementos en dos partes iguales y luego busca en ellas. Al haber niveles, hay $O(\log n)$ pero dentro de cada nivel, la búsqueda es $O(n)$.

$$O(n \log n)$$

Caso promedio:

↓

Supongamos un pivote al azar que tiene la probabilidad de caer en cualquier elemento, queda así: $n-1-q$.

Por cada nivel, la complejidad es $O(\log n)$. Sin embargo, para las comparaciones internas son $O(n)$

$$O(n \log n)$$

Peor caso

↓

El pivote es 0 o $n-1$. Al no haber niveles, no hay $O(\log n)$.

Pero al ser recursivo, queda $(n(n+1))/2$

$$O(n^2)$$

Problema 5: Determine si los siguientes enunciados son verdaderos o falsos. Justifique sus respuestas para recibir créditos completos.

a) Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = O(f(n))$

Verdadero

por transitividad de las funciones es verdadero

b) Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = \Omega(f(n))$

verdadero.

Si Ω es al mejor caso, por lo que, por transitividad es verdadero.

c) $f(n) = O(n^2)$, donde $f(n)$ está definido como el tiempo de ejecución del siguiente programa Python.

def A(n):

atupla = tuple(range(0, n))

S = set()

for i in range(0, n): $\leftarrow O(n)$

for j in range(i + 1, n): $\leftarrow O(n)$

S.add(atupla[i:j])

tamando en cuenta esto

Es falso, ya que $O(n^3)$ y no $O(n^2)$